

Aproximaciones a la implementación de Planificadores

Javier Vázquez

CS - FIB- UPC

1

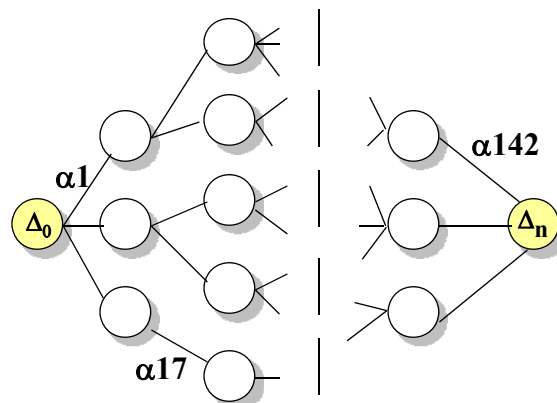
Aproximaciones a la planificación automática

- Aproximaciones más relevantes:
 - Planificación en el espacio de estados (*State-space planning*)
 - Planificación en el espacio de planes (*Plan-space planning* ó *PSP*)
 - Planificación jerárquica (*Hierarchical Task Network Planning* ó *HTN*)
- Otras técnicas de Planificación
 - Reutilización de Planes
 - Planificación específica de un dominio (*Domain-specific planning*)
- La competición internacional de planificación (ICAPS)

Planificación en el espacio de estados

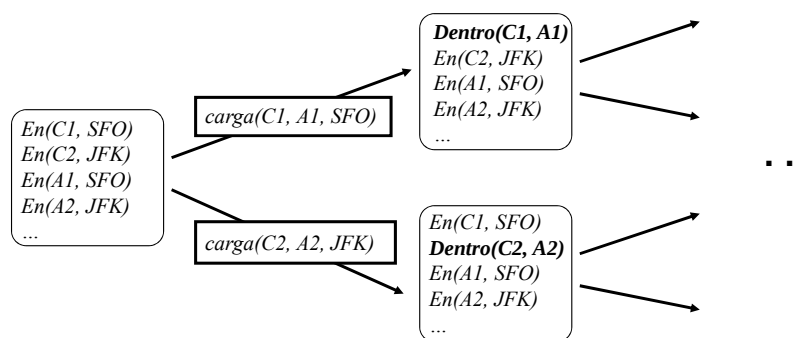
Planificación en el espacio de estados (State-Space Planning)

- Idea:
 - cada nodo representa un estado del mundo
 - El estado del mundo se define mediante un conjunto de predicados y variables
 - Un plan es un camino dentro del espacio de estados



Estrategias de planificación en espacio de estados(1)

- **Busqueda hacia delante** (*planificación progresiva*)
 - El estado inicial de la búsqueda es el **estado inicial del problema**
 - En cada momento se intenta unificar con las **precondiciones** de las acciones
 - Se cambia la descripción del estado añadiendo o eliminando literales de los **efectos** de las acciones



Planificación progresiva determinista

- Implementaciones deterministas de búsqueda hacia adelante :
 - Anchura prioritaria (*breadth-first search*)
 - Profundidad prioritaria (*depth-first search*)
 - best-first search (ej.: A*)
 - greedy best first
- Anchura prioritaria y A* son completas...
 - ... pero no suelen ser prácticas al necesitar demasiada memoria (exponencial en la longitud de la solución) → Memory Bound A*
- En la práctica se suelen usar profundidad prioritaria o greedy
 - Problema: no son completas
 - Pero la planificación clásica tiene un conjunto finito de estados
 - Profundidad prioritaria se puede hacer completa controlando los ciclos

Heurísticos en planificación progresiva determinista

- Durante muchos años los investigadores en planificación han buscado algoritmos generales, totalmente independientes del dominio
- Problema: ¿Cómo hacer una búsqueda heurística estilo A* sin incluir conocimiento del dominio?
 - Durante varios años nadie consiguió encontrar una buena función h
 - Solución: heurísticos que se calculaban automáticamente a partir de un grafo de los operadores y sus dependencias (GRAPHPLAN).
- Ej: FastForward [Hoffmann]

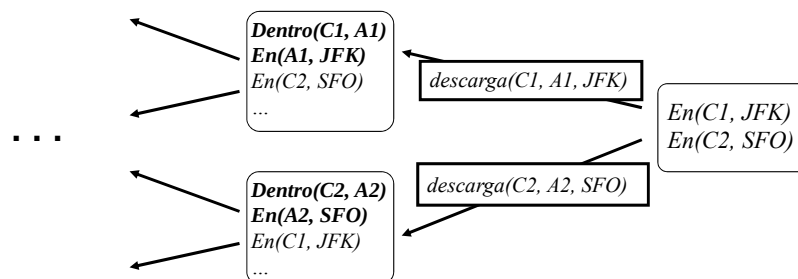
Problemas en la planificación progresiva determinista

- Problema: Cuando el factor de ramificación es muy elevado:
 - Existen muchas acciones aplicables que no nos llevan al objetivo
 - Las implementaciones deterministas pueden perder mucho tiempo probando múltiples acciones irrelevantes.
- Una posible solución: añadir heurísticos específicos del dominio
 - Lo veremos más adelante.

Estrategias de planificación en espacio de estados (2)

- **Busqueda hacia atrás** (*planificación regresiva*)

- El estado inicial de la búsqueda es el estado final del problema
- En cada momento se intenta unificar con los efectos de las acciones. Los efectos positivos se eliminan de la descripción.
- Se añaden los literales de las precondiciones excepto si ya aparecen en la descripción actual
- La búsqueda acaba cuando todas las precondiciones son satisfechas por el estado inicial del problema



Planificación regresiva: teoría subyacente (I)

- En la búsqueda hacia adelante, se empezaba en el estado inicial y se computaban transiciones de estados
 - nuevo estado $s' = \gamma(s, a)$
- En la búsqueda hacia atrás, empezamos en el objetivo y se computan transiciones inversas de estados
 - nuevo conjunto de subobjetivos $g' = \gamma^{-1}(g, a)$

Planificación regresiva: teoría subyacente (II)

- Para definir $\gamma^{-1}(g,a)$, hemos de definir primero el concepto de *relevancia*:
 - Una acción a es relevante para un objetivo g si
 - a hace al menos uno de los literales de g cierto
 - $g \cap \text{efectos}(a) \neq \emptyset$
 - a no hace falso ninguno de los literales de g
 - $g^+ \cap \text{efectos}^-(a) = \emptyset$ y $g^- \cap \text{efectos}^+(a) = \emptyset$
- Def: si a es relevante para g , entonces
 - $\gamma^{-1}(g,a) = (g - \text{efectos}(a)) \cup \text{precond}(a)$
 sino $\gamma^{-1}(g,a)$ esta indefinido
 - Ej: en el caso
 - $g = \{\text{on}(b1,b2), \text{on}(b2,b3)\}$
 - $a = \text{stack}(b4,b5)$
 - ¿Cual seria $\gamma^{-1}(g,a)$?

Problema de la planificación regresiva

- En PDDL no siempre tenemos todas las variables de la precondition en los efectos:

[ejemplo de los rovers de marte –rovers.zip-]

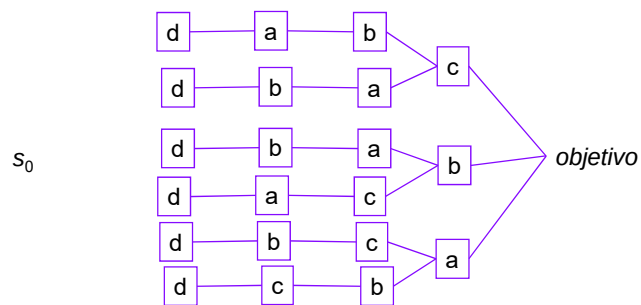
```
(:action drop
  :parameters (?x - rover ?y - store)
  :precondition (and (store_of ?y ?x) (full ?y) )
  :effect (and (not (full ?y)) (empty ?y))
)
```

- ¿Como definimos aquí el posible valor de $?x$ si no lo podemos unificar en los efectos?

$\gamma^{-1}(g,a) = (g - \text{efectos}(a)) \cup \text{precond}(a)$ dejará el valor de $?x$ indefinido

Problema de la planificación regresiva

- Aunque genera un espacio de búsqueda algo más pequeño, aún puede ser muy grande y algo ineficiente
- Ejemplo:
 - En el caso de tres acciones a , b y c independientes, una acción d que las ha de preceder siempre, y que no hay ningún camino desde s_0 al estado necesario como input de d
 - El algoritmo intenta todas las ordenaciones posibles de a , b y c antes de darse cuenta que no hay solución.



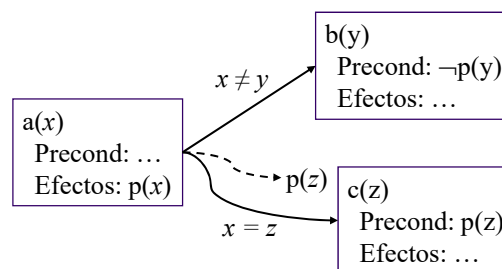
Planificación en el espacio de planes

Planificación en el espacio de planes (Plan-Space Planning)

- Idea:
 - Búsqueda hacia atrás desde el objetivo
 - Cada nodo del espacio de búsqueda es un plan parcial que incluye:
 - un conjunto de operadores parcialmente instanciados
 - un conjunto de restricciones sobre los operadores
- Proceso:
 - Descomponer conjuntos de objetivos en objetivos individuales
 - Planificar para cada uno de ellos por separado
 - Se van detectando y resolviendo los ‘fallos’ que hacen que aun no sea un plan, imponiendo más y más restricciones hasta que se tiene un plan parcialmente ordenado.
- Una extensión de planificación temporal en el espacio de planes se ha usado en los Mars Rovers de NASA.

Plan-Space Planning: restricciones

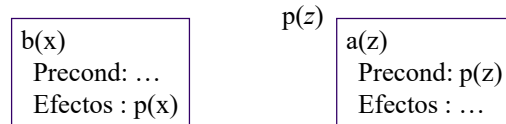
- 3 tipos de restricciones:
 - *restricciones de precedencia*: a debe preceder a b
 - *restricciones de asignación*:
 - restricciones de desigualdad: $x \neq y$
 - Restricciones de igualdad: $x = z$
 - *enlaces causales (causal links)*:
 - usar la acción a para obtener la precondición p necesaria para la acción c



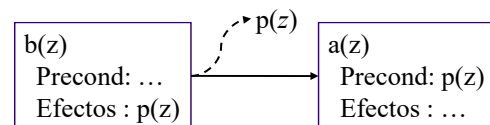
[Ejemplo de Dana Nau en Lecture slides for *Automated Planning*.]

Plan-Space Planning: Fallos - 1. Objetivos abiertos

- Objetivo abierto:
 - Una acción a tiene una precondición p que no hemos decidido como obtener



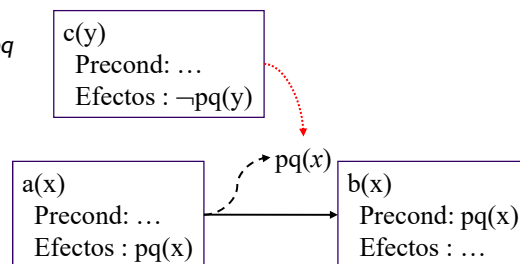
- Resolviendo el fallo:
 - Encontrar una acción b (ya en el plan o añadirla) que pueda usarse para obtener p
 - Puede preceder a y producir p
 - Instanciar variables y/o restringir las asignaciones de variables
 - Crear un enlace causal



[Ejemplos de Dana Nau en Lecture slides for Automated Planning.]

Plan-Space Planning: Fallos - 2. Ataques

- Ataque: una interacción que elimina condiciones
 - la acción a genera la precondición (e.g., $pq(x)$) de una acción b
 - otra acción c es capaz de eliminar la precondición pq
- Resolviendo el fallo:
 - Imponer una restricción para evitar que c elimine la precondición pq
 - haciendo que b preceda a c
 - Haciendo que c preceda a a
 - Restringir variables para prevenir que c elimine a pq



[Ejemplo de Dana Nau en Lecture slides for Automated Planning.]

Plan-Space Planning: El algoritmo PSP

```

PSP( $\pi$ )
   $flaws \leftarrow \text{OpenGoals}(\pi) \cup \text{Threats}(\pi)$ 
  if  $flaws = \emptyset$  then return( $\pi$ )
  select any flaw  $\phi \in flaws$ 
   $resolvers \leftarrow \text{Resolve}(\phi, \pi)$ 
  if  $resolvers = \emptyset$  then return(failure)
  nondeterministically choose a resolver  $\rho \in resolvers$ 
   $\pi' \leftarrow \text{Refine}(\rho, \pi)$ 
  return(PSP( $\pi'$ ))
end

```

- PSP es completo
- Devuelve un plan ordenado parcialmente
 - Cualquier orden total del plan satisfacerá los objetivos
 - Una ejecución paralela que mantenga el orden parcial también cumplirá los objetivos.

Plan-Space Planning: Ejemplo 1

- “Estoy en casa y necesito comprar leche, bananas y un taladro. La leche y las bananas se venden en el supermercado, el taladro en la ferretería”

- Operadores:

- **Start**

Precond: none

Effects: At(Home), sells(HWS,Drill), Sells(SM,Milk),
Sells(SM,Banana)

- **Finish**

Precond: Have(Drill), Have(Milk), Have(Banana), At(Home)

- **Go(l,m)**

Precond: At(l)

Effects: At(m), $\neg$ At(l)

- **Buy(p,s)**

Precond: At(s), Sells(s,p)

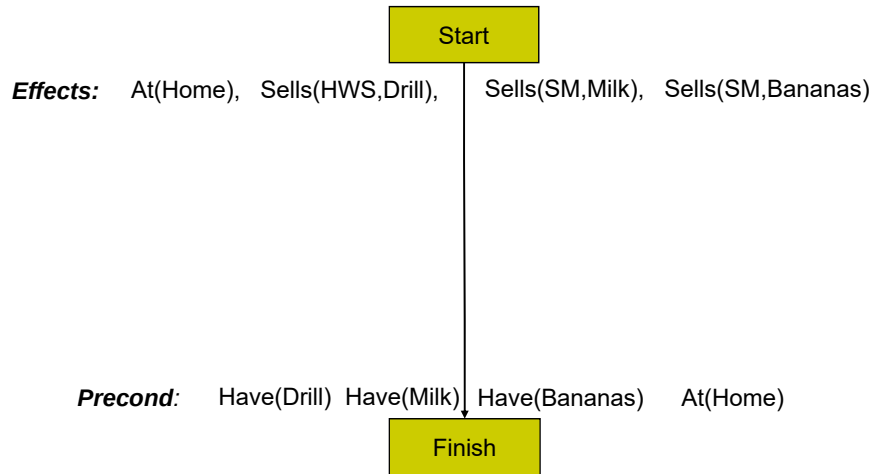
Effects: Have(p)

Start y Finish son acciones ficticias que usaremos en vez del estado inicial y el objetivo

[Ejemplo de Dana Nau en Lecture slides for *Automated Planning*.]

Plan-Space Planning: Ejemplo 1 (cont)

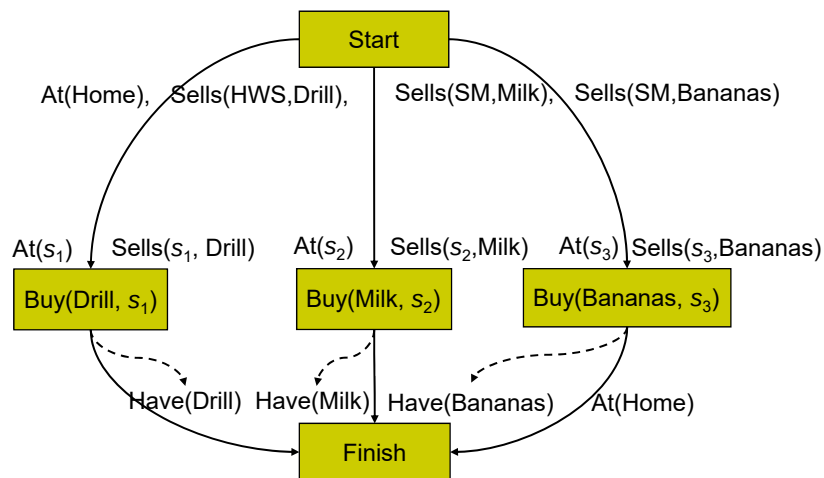
- Le damos a PSP un plan inicial π : **Start**, **Finish** y una restricción de orden.



[Ejemplo de Dana Nau en Lecture slides for *Automated Planning*.]

Plan-Space Planning: Ejemplo 1 (cont)

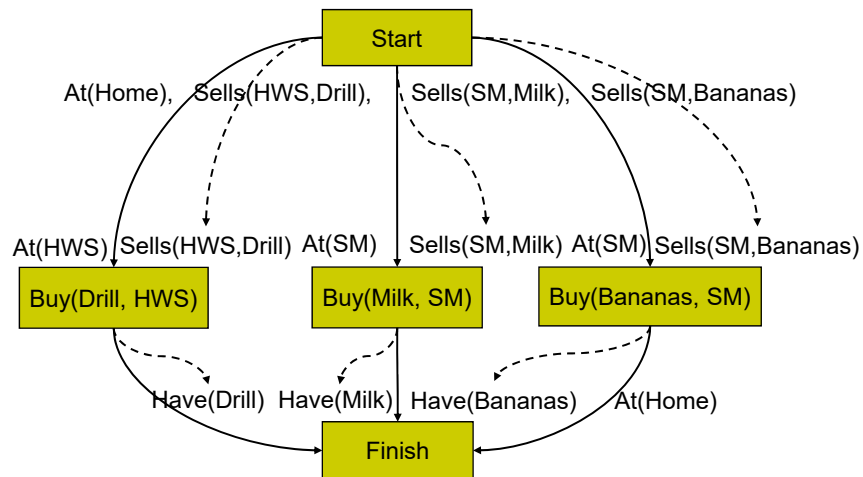
- Primeros tres refinamientos: las únicas formas de obtener las precondiciones de los Have



[Ejemplo de Dana Nau en Lecture slides for *Automated Planning*.]

Plan-Space Planning: Ejemplo 1 (cont)

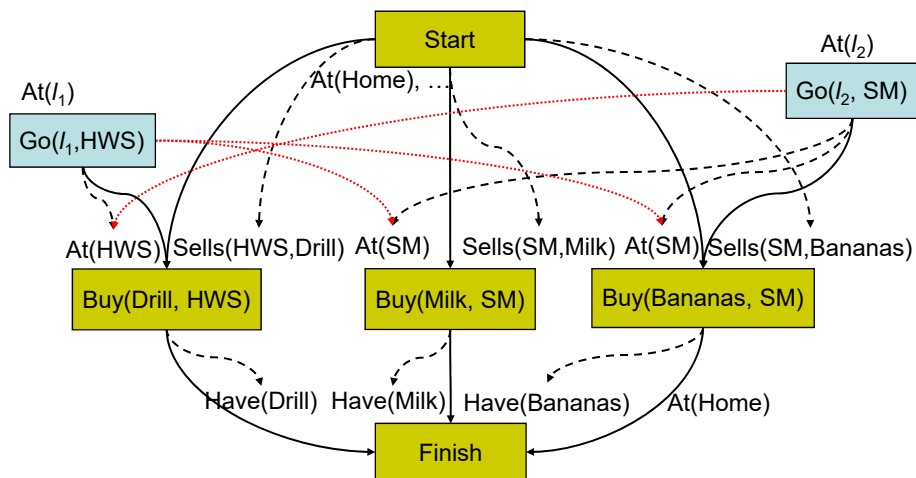
- Tres refinamientos más: las únicas formas de obtener las precondiciones de los Sells



[Ejemplo de Dana Nau en Lecture slides for Automated Planning.]

Plan-Space Planning: Ejemplo 1 (cont)

- Dos refinamientos más: las únicas formas de obtener $At(HWS)$ y $At(SM)$
 - Esta vez aparecen varios ataques



[Ejemplo de Dana Nau en Lecture slides for Automated Planning.]

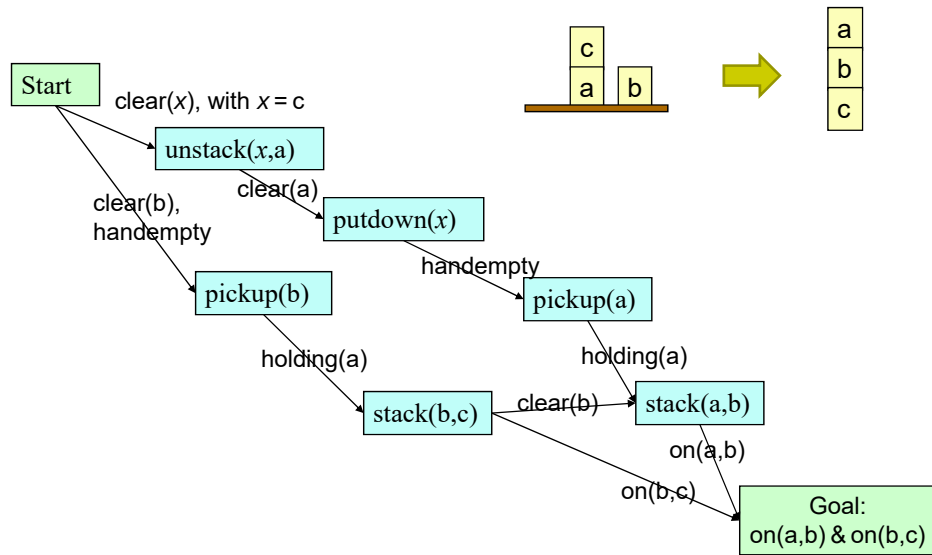
-
- The diagram illustrates a state transition graph for a planning problem. The nodes represent states, and the edges represent transitions. Solid arrows indicate actions, while dashed arrows indicate goals. The sequence of actions is: Start → Go(Home, HWS) → Buy(Drill, HWS) → Buy(Milk, SM) → Buy(Bananas, SM) → Go(I₃, Home) → Finish. The sequence of goals is: Start → At(HWS) → At(SM) → At(I₃) → At(Home). The graph shows how the state evolves through a series of actions and goals, leading to the final state 'Finish'.

Plan-Space Planning: Ejemplo 1 - Plan final

-
- The diagram illustrates a sequence of actions and states in a planning problem. The states are represented by yellow boxes, and the actions are represented by light blue boxes. The transitions are shown as directed edges, with solid lines for primary actions and dashed lines for secondary or conditional actions.
- Start** (yellow box) is the initial state.
 - Go(Home, HWS)** (light blue box) is an action that leads to **At(Home)** (text label) and **At(HWS)** (text label).
 - Buy(Drill, HWS)** (yellow box) is an action that leads to **Sells(HWS, Drill)** (text label) and **At(HWS)** (text label).
 - Sells(HWS, Drill)** (text label) leads to **At(SM)** (text label) and **Sells(SM, Milk)** (text label).
 - Buy(Milk, SM)** (yellow box) is an action that leads to **Sells(SM, Milk)** (text label) and **At(SM)** (text label).
 - Sells(SM, Milk)** (text label) leads to **At(SM)** (text label) and **Sells(SM, Bananas)** (text label).
 - Buy(Bananas, SM)** (yellow box) is an action that leads to **Sells(SM, Bananas)** (text label) and **At(SM)** (text label).
 - Sells(SM, Bananas)** (text label) leads to **At(SM)** (text label) and **Go(SM, Home)** (light blue box).
 - Go(SM, Home)** (light blue box) leads to **At(Home)** (text label) and **Finish** (yellow box).
 - Finish** (yellow box) is the final state.
- The diagram also includes several dashed arrows representing conditional or secondary transitions, such as from **Start** to **At(HWS)**, **Go(Home, HWS)** to **At(HWS)**, and **At(HWS)** to **At(SM)**.

15

Plan-Space Planning: Ejemplo 2



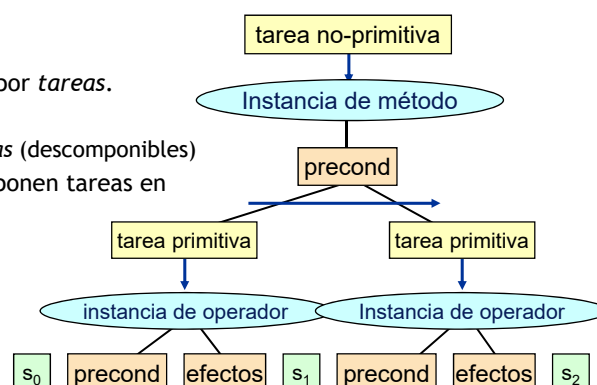
Planificación jerárquica

Planificación Jerárquica (Hierarchical Task Network Planning ó HTN)

- Hasta ahora todos los métodos que hemos visto trabajan en un único nivel de abstracción.
- Los humanos somos capaces de generar planes en diferentes niveles, desde planes de muy alto nivel a planes muy detallados.
- El plan de alto nivel a veces se llama receta y sirve como guía para la planificación a bajo nivel.
- **Planificación Jerárquica:**
 - Describe tareas y subtareas, y les asocia acciones.
 - Las tareas forman una red de tareas jerárquica (*Hierarchical Task Network* ó *HTN*)
 - El motor de planificación es capaz de explorar los diferentes niveles de (sub)tareas.

Planificación jerárquica simple

- Descripción del problema:
 - *Estados y operadores:* como en planificación clásica
 - No hay *objetivos*
 - Planificación guiada por *tareas*.
 - *Tareas primitivas*
 - *Tareas no primitivas* (descomponibles)
 - Los *métodos* descomponen tareas en sub-tareas

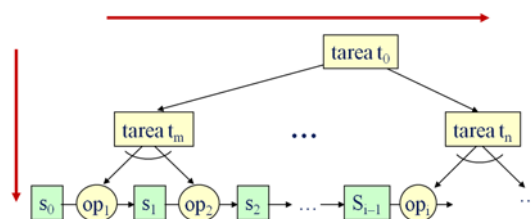


Planificación jerárquica simple: tareas y métodos

- **Tarea:** una expresión de la forma $t(u_1, \dots, u_n)$
 - t es un *símbolo de tarea*, y cada u_i es un término
 - Dos tipos de símbolos de tarea:
 - **primitivos:** tareas de las que sabemos cómo ejecutarlas directamente
 - El símbolo de tarea es un nombre de operador
 - **no-primitivos:** tareas que se han de descomponer en subtareas
 - usar *métodos*
- **Método:** una tupla
 - $m = (\text{nombre}(m), \text{tarea}(m), \text{precond}(m), \text{subtareas}(m))$
 - $\text{nombre}(m)$: una expresión de la forma $n(x_1, \dots, x_n)$
 - $x_1, \dots, x_n$ son parametros - símbolos de variables
 - $\text{tarea}(m)$: una tarea no-primitiva
 - $\text{precond}(m)$: precondiciones (literales)
 - $\text{subtareas}(m)$: una secuencia parcialmente ordenada de las tareas $\langle t_1, \dots, t_k \rangle$

Planificación jerárquica simple: dominio y problema

- **Dominio de planificación:** *métodos, operadores*
- **Problema de planificación:** *métodos, operadores, estado inicial, lista de tareas*
- **Solución:** cualquier plan ejecutable que se pueda generar por aplicar de forma recursiva
 - métodos para tareas no-primitivas
 - operadores para tareas primitivas



Ejemplo de HTN

- Un ejemplo simple de planificación:
 - Ir de casa al parque.

```

method travel-by-foot
  precondition:  $distance(x, y) \leq 2$ 
  task: travel( $a, x, y$ )
  subtasks: walk( $a, x, y$ )

method travel-by-taxi
  task: travel( $a, x, y$ )
  precondition:  $cash(a) \geq 1.5 + 0.5 \times distance(x, y)$ 
  subtasks:  $\langle call-taxi(a, x), ride(a, x, y), pay-driver(a, x, y) \rangle$ 

operator walk
  precondition:  $location(a) = x$ 
  effects:  $location(a) \leftarrow y$ 

operator call-taxi( $a, x$ )
  effects:  $location(taxi) \leftarrow x$ 

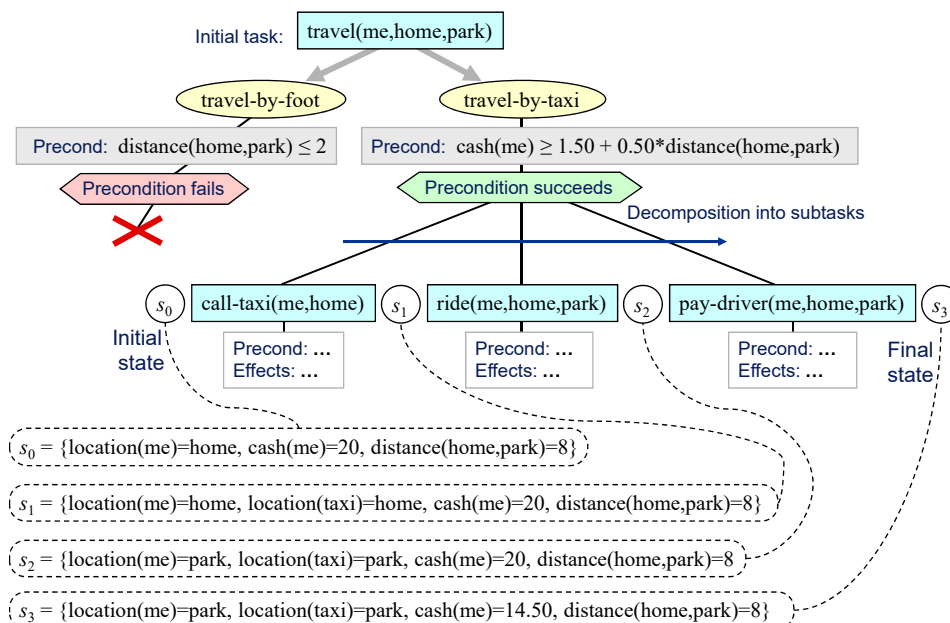
operator ride-taxi( $a, x$ )
  precondition:  $location(taxi) = x, location(a) = x$ 
  effects:  $location(taxi) \leftarrow y, location(a) \leftarrow y$ 

operator pay-driver( $a, x, y$ )
  precondition:  $cash(a) \geq 1.5 + 0.5 \times distance(x, y)$ 
  effects:  $cash(a) \leftarrow cash(a) - 1.5 + 0.5 \times distance(x, y)$ 

```

[Ejemplo de Dana Nau en Lecture slides for Automated Planning.]

Ejemplo de HTN: estoy en casa con \$20, quiero ir a un parque a 8 millas de aquí.



[Ejemplo de Dana Nau en Lecture slides for Automated Planning.]

Otras técnicas de Planificación

Otros resultados interesantes en planificación

- Reutilización de Planes (Plan Reuse)
- Planificación específica de un dominio (Domain-specific planning)
 - Heurísticos específicos de un dominio
 - Algoritmos específicos de un dominio

Reutilización de planes (Plan Reuse)

- Si planificar es algorítmicamente tan complejo...
- Idea: Reutilizar planes anteriores para resolver nuevos problemas de planificación
- Consiste en dos pasos
 - Reconocimiento (matching) del plan
 - Modificación del plan
- Resultado: según varios estudios...
 - En general, reutilizar planes suele ser aún más complejo a nivel algorítmico que planificar desde cero.
 - El cuello de botella es el plan matching
 - Da mejores resultados solo cuando dos problemas son lo suficientemente similares

Domain-specific planning - Heurísticos específicos de un dominio

- Originalmente se usaban heurísticos independientes del dominio
- Idea: ¿Y si se usan heurísticos dependientes del problema?
 - Problema: no podremos aplicarlo a otros problemas
- Ejemplo:
 - En el N-puzzle, la estimación detallada de la distancia entre el estado actual y el estado objetivo puede acelerar bastante la búsqueda del plan
- Resultado: normalmente los planificadores con heurísticos específicos del dominio mejoran su rendimiento respecto a los que son independientes del dominio

Domain-specific planning - Algoritmos específicos de un dominio

- Idea: podemos optimizar el algoritmo de planificación para resolver un problema concreto
- Ejemplo: Blocks World

```

while true
  if there is a clear block x such that
    x needs to be moved and
    x can be moved to a place where it won't need to be moved
  then move x to that place
  else if there is a clear block x such that
    x needs to be moved
  then move x to the table
  else if the goal is satisfied
  then return the plan
  else return failure
end_while

```

[Ejemplo de Dana Nau en Lecture slides for *Automated Planning*.]

- Resultado: normalmente estos planificadores son mucho más rápidos y eficientes (respecto a los planificadores que son independientes del dominio), pero el código de estos planificadores es muy difícil de reusar

La competición internacional de planificación

La competición internacional de planificación

- Una forma de hacer que la tecnología de planificación avance más rápido
- Primera edición en 1998
- Creadora del lenguaje PDDL
- Efectivamente ha hecho evolucionar el área de investigación
 - Explosión de nuevos métodos
 - Hibridación de métodos
 - Optimización de métodos
- Todos los resultados están disponibles en la web
<http://ipc.icaps-conference.org/>
- El código de muchos de los planificadores esta disponible

